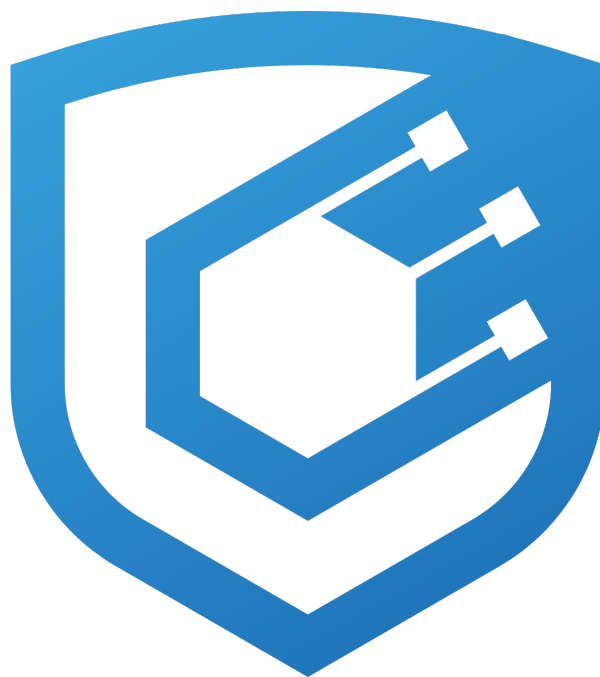




Securitize Whitelist ACL Support Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Farouk](#)

[Al-qaqa](#)

June 2, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	3
6	Executive Summary	3
7	Findings	6
7.1	Low Risk	6
7.1.1	spl_token_whitelist::change_admin is single-step and accepts any Pubkey including the default zero key	6
7.2	Informational	8
7.2.1	Hardcoded MINT_CONFIG_DISCRIMINATOR = 1 duplicates upstream TACL state with no compile-time tie-back	8
7.2.2	SPL sRFC-37 access-control thaw forwards caller-selected CPI accounts without tying them to the validated authority path	8
7.2.3	getWhitelistIxS is not checking the ownership of the tokenAccounts passed with tx owner .	10
7.2.4	Whitelisted event does not record which freeze-authority path was used	11

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The Securitize SVM Whitelist program is a Solana Anchor-based access-control system that onboards regulated investors into permissioned SPL Token-2022 tokens issued under the Securitize platform. It governs the moment an investor's token account transitions out of the default-frozen state into a transfer-capable state and emits a `Whitelisted` event after the thaw, providing the durable on-chain signal that downstream protocol logic and off-chain systems use to recognize the investor as onboarded for that mint.

Onboarding flows through a single `whitelist` instruction that thaws one or more token accounts belonging to the investor in the same call. The mint carries the SPL Token-2022 `DefaultAccountState` extension set to `Frozen`, so newly initialized token accounts for that mint start in the frozen state and cannot hold balance or participate in transfers until the whitelist program thaws them. When the investor invokes `whitelist`, the program identifies the appropriate freeze authority for the mint, then for each supplied token account validates that the account is owned by the signer, references the expected mint, is owned by the Token-2022 program, and is currently frozen — and only then dispatches the thaw. The instruction is all-or-nothing: if any per-account validation fails or any thaw CPI reverts, the entire instruction reverts and no `Whitelisted` event is emitted.

This audit covers a delta that adds support for tokens whose Mint and Freeze authorities are held by a separate Token ACL Program PDA rather than by the whitelist program itself. Because SPL Token-2022 binds each mint to a single freeze authority, mints migrated to the ACL authority model cannot be thawed directly by the whitelist program. The updated `whitelist` instruction inspects the mint's on-chain freeze authority and routes the thaw accordingly: legacy mints follow a direct Token-2022 freeze invocation, while ACL-mode mints reach Token-2022 through a CPI into the ACL Program, with an additional sRFC-37 access-control branch that interacts with the upstream `spl-token-access-control` program. The delta also reshapes the whitelist instruction's account list: the signer field previously named `freezeAuthority` in the IDL/SDK is renamed to `authority`, reflecting its role as the CPI authority rather than the mint's actual freeze authority; a new `freeze_authority` unchecked account is added alongside it so the program can read the mint's freeze authority directly from on-chain state when routing; and the SDK is bumped to `v0.1.0` to reflect the breaking interface change.

The program is organized as two sibling Anchor programs — `spl-token-whitelist` for SPL Token-2022 mints and `dstoken-whitelist` for the legacy DSToken flow — sharing the same administrative model: a state PDA initialized by the upgrade authority and rotated by a single-step `change_admin` instruction, with `pause` and `unpause` gating the

permissionless whitelist path during incident response. Authority detection is implemented as a typed classification (`FreezeAuthorityType`) that derives the expected ACL authority PDA from the mint, compares it against the mint's on-chain freeze authority field, and produces a dispatch value that drives both the thaw CPI selection and the set of accounts forwarded to it. Trusted dependencies include the ACL Program (audited separately by Cyfrin in March 2026), the upstream `spl-token-access-control` program, and the SPL Token-2022 runtime.

For the system to remain safe, authority detection must classify each mint exclusively from validated on-chain account state, no instruction path may emit the `Whitelisted` event without first executing a successful thaw under the correct authority routing for every supplied token account, and every account forwarded into a downstream CPI must be tied back to a derivation the wrapper itself has validated.

5 Audit Scope

The audit scope was limited to:

```
// see https://github.com/securitize-io/bc-solana-whitelist-sc/pull/1/changes
// for full list of changes

// primary scope - very minor changes
programs/dstoken-whitelist/src/instructions/admin/initialize.rs
programs/spl-token-whitelist/src/instructions/admin/change_admin.rs
programs/spl-token-whitelist/src/instructions/admin/initialize.rs
programs/spl-token-whitelist/src/constants.rs

// primary scope - new files
programs/spl-token-whitelist/src/instructions/cpi/srfc37/thaw_account.rs
programs/spl-token-whitelist/src/utils/freeze_authority_type.rs

// primary scope - significant changes to existing files
programs/spl-token-whitelist/src/instructions/whitelist.rs

// secondary scope - mostly minor changes but auditors
// need to ensure that these match correctly with smart contract changes
packages/spl-token-whitelist-sdk/src/instructions/whitelist.ts
packages/spl-token-whitelist-sdk/src/idl/securitize-solana-spl-whitelist-idl.json
packages/spl-token-whitelist-sdk/src/types/base-types.ts
packages/spl-token-whitelist-sdk/src/types/constants.ts
packages/spl-token-whitelist-sdk/src/types/securitize-solana-spl-whitelist-types.ts
packages/spl-token-whitelist-sdk/src/spl-token-whitelist.ts
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [Whitelist ACL Support](#) code provided by [Securitize](#). The findings consist of 1 Low severity issue with the remainder being informational.

The audit reviewed a delta to the Securitize SVM Whitelist program that introduces ACL-Program authority routing for SPL Token-2022 mints, alongside a breaking field rename (`freezeAuthority` → `authority`) in the whitelist entry instruction interface. The reviewed surface is small and well-scoped: the new authority classification helper, the `SRFC-37` thaw CPI branch, the modified `whitelist` handler, the two `initialize` paths, the SPL `change_admin` handler, and the corresponding TypeScript SDK and IDL artifacts. Out-of-scope components — the ACL Program, `spl-token-access-control`, the SPL Token-2022 runtime, and off-chain backend services — were treated as trusted under the engagement's threat model.

Review effort concentrated on the priority areas defined in the audit context: authority detection performed strictly from on-chain mint state, ACL Program account validation against a hardcoded program ID, token-account-ownership enforcement against the instruction signer, CPI error propagation so that a failed thaw cannot result in a `Whitelisted` event for an account that remains frozen, and deserialization compatibility for the

renamed `authority` field against previously-stored on-chain state. The legacy direct-thaw path was checked for regression, and the edge cases enumerated in the audit context — zero-pubkey freeze authority, already-thawed accounts, duplicate whitelisting, partial CPI account lists, and ACL-program substitution — were exercised against the implementation. The TypeScript SDK that constructs the `additionalAccounts` array for each thaw path was reviewed alongside the on-chain handlers to verify the two sides agree on account ordering and ownership-based path selection.

The delta is, on the whole, a focused and disciplined extension. Authority detection routes exclusively on the mint's on-chain freeze authority field, ACL-program substitution is blocked by hardcoded program-ID equality checks for both `spl-token-access-control` and Token ACL, token account ownership is enforced before either thaw path executes, and CPI errors propagate cleanly so that no `Whitelisted` event can be emitted while any supplied token account remains frozen. The open findings cluster into defense-in-depth and operational-hygiene themes spanning the on-chain handlers, the off-chain SDK, and event observability. On the on-chain side: an admin-rotation handler that is single-step and accepts the default zero pubkey, leaving room for an irrecoverable misconfiguration if the admin signs a rotation to a key they do not control; an sRFC-37 thaw branch that forwards three caller-selected CPI account slots without locally tying them back to PDAs the wrapper has already derived from the mint, which is currently safe only because the upstream program revalidates them through its own Anchor constraints; a hardcoded `MINT_CONFIG_DISCRIMINATOR` literal that duplicates an upstream state byte with no compile-time tie-back, creating drift risk if the upstream layout ever changes; and a `Whitelisted` event that does not record the resolved freeze-authority path, leaving off-chain consumers blind to which thaw route was taken. On the off-chain SDK side: the `get_remaining_accounts` helper's `AcProgram` routing branch keys on raw PDA equality without first verifying that the freeze-authority account is owned by the System Program, diverging from the on-chain handler's ownership-first ordering and producing client-side misclassification that the on-chain handler would otherwise catch at execution; and `getWhitelistIdxs` accepts caller-supplied token accounts without verifying off-chain that they are owned by the transaction signer, deferring rejection to the on-chain handler rather than failing fast at construction time.

Centralization Risks

The whitelist program is upgradeable, and a single admin role gates the `pause` and `unpause` kill-switch protecting the permissionless `whitelist` instruction. Admins do not move user funds directly, but they do control the operational availability of onboarding, and the upgrade authority can replace the program with arbitrary logic. By the regulated nature of the broader Securitize platform, users must extend trust to the admin and upgrade authorities to operate the program in good faith; conversely, this same upgradeability means that issues discovered post-deployment can in most cases be remediated by redeploying the affected program.

Protocol Invariants

- Authority routing for `whitelist` is determined exclusively from the mint's on-chain freeze authority field, never from caller-supplied parameters or instruction data.
- Any account supplied as the ACL Program or the Token ACL program in the instruction must equal the expected hardcoded program ID before any CPI is invoked.
- Each token account being thawed must belong to the investor signing the instruction.
- The `Whitelisted` event must be emitted only after every supplied token account has been successfully thawed, and a reverted thaw — direct or via CPI — must revert the entire instruction with no partial state.
- The legacy direct-thaw path must behave identically to its pre-change implementation for mints whose freeze authority is the whitelist program itself.
- The `freezeAuthority` → `authority` field rename must not introduce deserialization mismatches for accounts serialized under the prior layout.
- The TypeScript SDK's instruction-construction helpers must mirror the on-chain handler's preconditions — routing-branch ordering, ownership of caller-supplied token accounts, and additional-account layout per freeze-authority mode — so that off-chain construction fails fast rather than producing transactions that the on-chain handler will reject.

Summary

Project Name	Whitelist ACL Support
Repository	bc-solana-whitelist-sc
Commit	fcc2a5df4bc5...
Audit Timeline	May 18th - May 20th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Informational	4
Gas Optimizations	0
Total Issues	5

Summary of Findings

[L-1] <code>spl_token_whitelist::change_admin</code> is single-step and accepts any Pubkey including the default zero key	Acknowledged
[I-1] Hardcoded <code>MINT_CONFIG_DISCRIMINATOR = 1</code> duplicates upstream TACL state with no compile-time tie-back	Acknowledged
[I-2] SPL sRFC-37 access-control thaw forwards caller-selected CPI accounts without tying them to the validated authority path	Acknowledged
[I-3] <code>getWhitelistIxs</code> is not checking the ownership of the tokenAccounts passed with tx owner	Resolved
[I-4] <code>Whitelisted</code> event does not record which freeze-authority path was used	Acknowledged

7 Findings

7.1 Low Risk

7.1.1 `spl_token_whitelist::change_admin` is single-step and accepts any Pubkey including the default zero key

Description: `change_admin_handler` writes the caller-supplied `new_admin` into `SplWhitelistState::admin` after only checking `state.admin != new_admin`. There is no `require_keys_neq!(new_admin, Pubkey::default(), ...)` filter, no validation that `new_admin` is a Signer in the same transaction, and no two-step propose-then-accept handshake. Because every other admin-gated instruction (`pause`, `unpause`, future `change_admin`) is bound by `has_one = admin`, a rotation to the all-zero pubkey or to any address whose private key is not held by the protocol team permanently locks those instructions - no further signer can satisfy `has_one = admin`, and the upgrade authority is not re-consulted after `initialize`.

```
/// Updates the admin pubkey stored in the whitelist state.
pub fn change_admin_handler(ctx: Context<ChangeAdmin>, new_admin: Pubkey) -> Result<> {
    let spl_whitelist_state = &mut ctx.accounts.spl_whitelist_state;

    require!(
        spl_whitelist_state.admin != new_admin,
        SplWhitelistErrorCode::NoChange
    );

    spl_whitelist_state.admin = new_admin;

    emit!(crate::events::AdminChanged {
        admin: ctx.accounts.admin.key(),
        new_admin,
    });

    Ok(())
}
```

The same shape exists in the `dstoken-whitelist` program's `change_admin` handler (out of formal audit scope; included for context - the mitigation should be applied symmetrically across both programs).

Impact: If the admin signs a `change_admin` instruction with `new_admin = Pubkey::default()` (a fat-finger encoding mistake, an SDK / CLI argument-construction bug, a JSON deserialization that defaults missing fields to zero) or with any address whose key the team cannot produce on a later signature, the whitelist state PDA becomes administratively bricked. `pause` and `unpause` can no longer be called, so the only operational kill-switch protecting the permissionless whitelist instruction is lost. A subsequent `change_admin` to recover from the mistake is also blocked, since the current `admin` field is unsignable. The only remaining recovery is a program redeploy by the upgrade authority that rewrites `change_admin_handler` (or that re-initializes the state PDA through a migration). The harm requires admin error or key compromise to manifest, which caps severity at Low, but recovery is non-trivial and operationally disruptive.

Recommended Mitigation: Add a zero-key filter and adopt a two-step rotation in both `change_admin` handlers. Concretely, in `programs/spl-token-whitelist/src/instructions/admin/change_admin.rs` (and apply the same change symmetrically to the `dstoken-whitelist` `change_admin` handler, which is out of formal audit scope):

1. Reject the default pubkey at the top of the handler:

```
require_keys_neq!(
    new_admin,
    Pubkey::default(),
    SplWhitelistErrorCode::InvalidAdmin
);
```

2. Split the rotation into a propose step (the current admin writes the proposed admin key into a new pending-

admin field on the state PDA) and an accept step that requires the new admin to sign the second transaction and clears the pending-admin field upon acceptance. This guarantees that the destination key is controlled by a live signer before it replaces the active admin. If the propose/accept refactor is out of scope for this release, at minimum require the `new_admin` to sign the same `change_admin` transaction by adding a second `Signer<'info>` account to the `ChangeAdmin` struct and constraining it so that the signer's key equals `new_admin`.

Securitize: Acknowledged.

7.2 Informational

7.2.1 Hardcoded MINT_CONFIG_DISCRIMINATOR = 1 duplicates upstream TACL state with no compile-time tie-back

Description: `programs/spl-token-whitelist/src/constants.rs:7` declares `pub const MINT_CONFIG_DISCRIMINATOR: u8 = 1` as a hardcoded literal. The same byte value is the discriminator field on the upstream `spl_token_access_control` (TACL) program's `MintConfig` state struct (which is out of audit scope and not redistributed as a consumable public constant by the upstream crate). The in-scope code re-declares the byte locally and compares the deserialized struct's discriminator against the local literal at `programs/spl-token-whitelist/src/utils/freeze_authority_type.rs:77`:

```
require!(
    mint_config.discriminator == MINT_CONFIG_DISCRIMINATOR,
    SplWhitelistErrorCode::InvalidMintConfig
);
```

The discriminator is sourced from upstream state but is not tied back to upstream at compile time. The two values can desynchronize across releases: if the upstream TACL program ever bumps `MintConfig::DISCRIMINATOR` (e.g. to 2 during a struct-layout migration), the in-scope `AcProgramWithSrfc37` and `SystemAccountWithSrfc37` thaw paths will start returning `InvalidMintConfig` for every legitimately-derived `MintConfig` account, silently DoS'ing the sRFC-37 thaw flows until the in-scope program is rebuilt and redeployed with a refreshed literal. The bug surfaces only after the upstream upgrade lands; the in-scope code compiles fine in isolation because the duplicated constant carries no dependency on the upstream definition.

The recoverability question is non-trivial because the program is upgradeable per the deployment context, but the failure mode is silent until users encounter it. A whitelist instruction that previously thawed accounts will start failing for a class of mints that was working a day earlier, with no signal in the in-scope code that explains why.

Recommended Mitigation: Two options, in order of preference:

1. Upstream fix - request that `spl_token_access_control` export `MintConfig::DISCRIMINATOR` (or the `MintConfig` discriminator byte) as a `pub const` consumable downstream, then replace the local literal with the imported constant. The Rust compiler will then enforce equality across the workspace via the dependency pin.
2. If upstream cannot be modified, add a build-time or test-time assertion that bridges the gap - e.g. a unit test that constructs a `MintConfig` via the upstream crate's public constructor (or a zeroed instance with the discriminator field set) and asserts the local `MINT_CONFIG_DISCRIMINATOR` equals what the upstream produces. This converts a silent runtime DoS after an upstream upgrade into a build-time failure during dependency bumping, surfacing the drift before deployment.

Securitize: Acknowledged.

7.2.2 SPL sRFC-37 access-control thaw forwards caller-selected CPI accounts without tying them to the validated authority path

Description: For an sRFC-37 mint whose Token ACL `MintConfig.freeze_authority` is the `spl-token-access-control` authority PDA, `FreezeAuthorityType::from_mint_and_freeze_authority` re-derives the access-control authority PDA from the mint, verifies it matches the supplied `srfc37_authority`, and returns `AcProgramWithSrfc37`:

```
let ac_program_authority = Pubkey::find_program_address(
    &[
        spl_token_access_control::constants::ACCESS_CONTROL_AUTHORITY_SEED,
        mint.key().as_ref(),
    ],
    &spl_token_access_control::ID,
)
.0;
if ac_program_authority == srfc37_authority.key() {
    require!(
```

```

        authority.key() != srfc37_authority.key(),
        SplWhitelistErrorCode::InvalidFreezeAuthorityType
    );
    return Ok(Self::AcProgramWithSrfc37);
}

```

The wrapper has authoritatively established, from the mint alone, the identity of the access-control authority for this thaw.

The `AcProgramWithSrfc37` branch of `thaw_account` then takes the next five entries of `remaining_accounts` and forwards them as the downstream CPI accounts:

```

Self::AcProgramWithSrfc37 => {
    require!(
        additional_accounts.len() == self.additional_accounts_len(),
        SplWhitelistErrorCode::InvalidFreezeAuthorityType
    );

    let access_control_authority = &additional_accounts[0];
    let access_control_state = &additional_accounts[1];
    let event_authority = &additional_accounts[2];
    let program = &additional_accounts[3];
    require!(
        *program.key == spl_token_access_control::ID,
        SplWhitelistErrorCode::InvalidFreezeAuthorityType
    );
    let token_acl_program = &additional_accounts[4];
    require!(
        *token_acl_program.key == TOKEN_ACL,
        SplWhitelistErrorCode::InvalidFreezeAuthorityType
    );

    let ac_program_thaw_accounts =
        spl_token_access_control::cpi::accounts::ThawAccount {
            access_control_authority: access_control_authority.to_account_info(),
            access_control_state: access_control_state.to_account_info(),
            authority: authority.to_account_info(),
            event_authority: event_authority.to_account_info(),
            ...
        };
}

```

The slot-to-CPI mapping is:

Index	Slot in CPI	Wrapper-side check
[0]	<code>access_control_authority</code>	none
[1]	<code>access_control_state</code>	none
[2]	<code>event_authority</code>	none
[3]	<code>spl-token-access-control program</code>	program ID equality
[4]	<code>Token ACL program</code>	program ID equality

The first three slots are accepted unconditionally. The wrapper does not require `additional_accounts[0] == srfc37_authority` (which it already derived), does not re-derive `[ACCESS_CONTROL_STATE_SEED, mint]` for slot [1], and does not re-derive the `#[event_cpi]` event authority PDA for slot [2].

This is currently safe only because the pinned `spl-token-access-control` revalidates these accounts through its own Anchor PDA constraints on `ThawAccount`:

```
#[account(
    seeds = [ACCESS_CONTROL_STATE, mint.key().as_ref()],
    bump = access_control_state.bump,
)]
pub access_control_state: Account<'info, AccessControlState>,

#[account(
    seeds = [ACCESS_CONTROL_AUTHORITY_SEED, mint.key().as_ref()],
    bump,
)]
pub access_control_authority: AccountInfo<'info>,
```

The wrapper has performed the classification work to know what the correct accounts must be, then discards that knowledge and relies on downstream constraints to catch mismatches.

This is the only freeze-authority mode with the gap: `SystemAccount` and `AcProgram` pass the Anchor-validated `freeze_authority` directly into the CPI rather than reading authority-bearing slots from `remaining_accounts`, and `SystemAccountWithSrfc37` only forwards the Token ACL program account (which is checked against `TOKEN_ACL`).

Recommended Mitigation: In the `AcProgramWithSrfc37` branch, derive `access_control_authority`, `access_control_state`, and the `__event_authority` PDA from the mint under `spl_token_access_control::ID`, and require `additional_accounts[0..3]` to equal those derivations before constructing the CPI. Apply the same change to the `AcProgram` branch's `access_control_state` and `event_authority` slots.

Securitize: Acknowledged.

7.2.3 `getWhitelistIx` is not checking the ownership of the tokenAccounts passed with tx owner

Description: When constructing a whitelist transaction, the off-chain `getWhitelistIx` function is called. The caller can either provide token accounts directly via `params.tokenAccounts`, or omit them — in which case the function automatically derives the owner's Associated Token Accounts (ATAs).

The problem arises when token accounts are provided explicitly. In this case, there is no off-chain validation to verify that the provided token accounts are actually owned by the transaction owner, allowing arbitrary token accounts to be passed without any client-side rejection.

`packages/spl-token-whitelist-sdk/src/instructions/whitelist.ts#getWhitelistIx`

```
export async function getWhitelistIx( ... ): Promise<TransactionInstruction[]> {
    ...
    let tokenAccounts: PublicKey[];

    if (params.tokenAccounts && params.tokenAccounts.length > 0) {
    >> tokenAccounts = params.tokenAccounts;
    } else { ... }

    ...

    return [...preIx, ix];
}
```

While the on-chain program does enforce that each token account's owner matches the transaction signer, the absence of this check on the client side means that a caller can construct and submit a transaction referencing token accounts they do not own. The transaction will be rejected on-chain, but the SDK silently allows the construction and submission of such invalid transactions without providing any meaningful client-side error, leading to a poor developer experience and potentially unexpected runtime failures.

`programs/spl-token-whitelist/src/instructions/whitelist.rs#whitelist_handler`

```
pub fn whitelist_handler<'info>(ctx: Context<'_, '_, '_, 'info, Whitelist<'info>>) -> Result<()> {
    ...
    // Thaw the token accounts
```

```

    for token_account_info in token_accounts_infos.iter() {
        ...
        require!(
>>         token_account.owner == ctx.accounts.owner.key(),
            SplWhitelistErrorCode::InvalidOwner
        );
        ...
    }
    ...
}

```

Recommended Mitigation: The `getWhitelistIdxs` function should validate that each explicitly provided token account is owned by the transaction owner before constructing the instruction. This check should be performed by fetching each token account's data and verifying that its `owner` field matches the provided owner address.

Securitize: Fixed in [db2c3c2](#).

Cyfrin: Verified.

7.2.4 Whitelisted event does not record which freeze-authority path was used

Description: The `whitelist` instruction resolves a `FreezeAuthorityType` before thawing accounts. That enum drives validation, how many remaining_accounts are consumed, and which CPI path runs (`token_2022::thaw_account`, `access-control thaw_account`, or `SRFC37 thaw_account`):

programs/spl-token-whitelist/src/utils/freeze_authority_type.rs

```

pub enum FreezeAuthorityType {
    SystemAccount,
    AcProgram,
    AcProgramWithSrfc37,
    SystemAccountWithSrfc37,
}

```

After processing, the program emits `Whitelisted` with only `owner`, `mint`, and `accounts_count`:

programs/spl-token-whitelist/src/events.rs

```

/// Emitted after thawing a batch of token accounts for an owner.
#[event]
pub struct Whitelisted {
    pub owner: Pubkey,
    pub mint: Pubkey,
    pub accounts_count: u8,
}

```

Impact:

- The resolved `freeze_authority_type` is not included in the event, preventing off-chain and monitoring systems from knowing the type of thaw/unfreezing that occurred.

Recommended Mitigation: We should include the type of `freeze_authority_type` in the event so that we know by which method we unfroze tokenAccounts

Securitize: Acknowledged.